Summer 2026
WLIURA Report

Topological Signatures of Cell Shape for Unsupervised Classification

Author: Ploy Phusri
Supervisor: Dr. Leah Edelstein-Keshet and Dr. Dhananjay Bhaskar
September 7, 2026

1 Background

This summer term, I worked with Dr. Leah Edelstein-Keshet and Dr. Dhananjay Bhaskar on a project: building a shape descriptor for segmented cell images using *topological data analysis* (TDA). We developed an unsupervised cell classification tool that provides quantitative visualization of perturbation effects. This tool can be applied to differentiate healthy from diseased cells.

Cell shape is a rich, label-free readout of a cell's state and environment, but the common way of storing shape as a raw 2D pixels is very sensitive to imaging noise and small deformations. Our goal was to come up with a descriptor represents 2D segmented cell shape that is interpretable and maintain translation and rotation invariants that is compatible with unsupervised classification pipeline. In this report I describe the motivation and the pipeline I implemented. This project is inspired by Dr. Dhananjay Bhaskar's Master Thesis (University of British Columbia, 2017). [**?**]

2 A Topological Descriptor of Cell Shape

2.1 Introduction

Topological Data Analysis (TDA) uses persistent homology and sub-level set filtration to extract the underlying topological features of a dataset. Persistent homology identifies and tracks structural properties—such as connected components, loops, and voids—across multiple scales. We extract these features using a sub-level set filtration, which constructs a sequence of nested spaces by progressively increasing a threshold parameter from a low to a high value, allowing us to observe how structural features evolve. This method is a good fit for cell morphology since cell shapes are noisy, variable, and non-rigidly deformed during imaging. Features that survive over a large range are treated as real structure, while features that appear and vanish quickly are treated as noise. We tested the descriptor on immune cells stimulated with two agents (PMA and LPS), each cultured on substrates of four stiffnesses (1, 10, 20, and 100 kPa, roughly 400 cells per stimulus), and asked whether cell *shape alone* can recover the stiffness of the substrate a cell was grown on.

2.2 Methods

Alignment. From each segmented binary mask I first compute a standardized cell: the mask is cropped to its bounding box, rotated so that its major axis (from `scikit-image`'s `regionprops`) [?] is horizontal with the heavier half on the left to preserve chirality, and re-centered on its centroid. This makes the radial and angular descriptors invariant to position and orientation.

Three descriptors. From the aligned cell I extract three complementary shape descriptors (Figure 1). The *radial distribution function* (RDF) is a histogram of cell pixels by distance from the centroid; the *angular distribution function* (ADF) is a histogram of cell pixels by angle about the centroid over $[0, 2\pi)$; and the *medial axis* descriptor extracts a morphological skeleton from the cell's boundary, storing the local thickness (the largest inscribed-circle radius) along this central axis. Because this is a purely geometric representation, it bypasses the need for an alignment step.

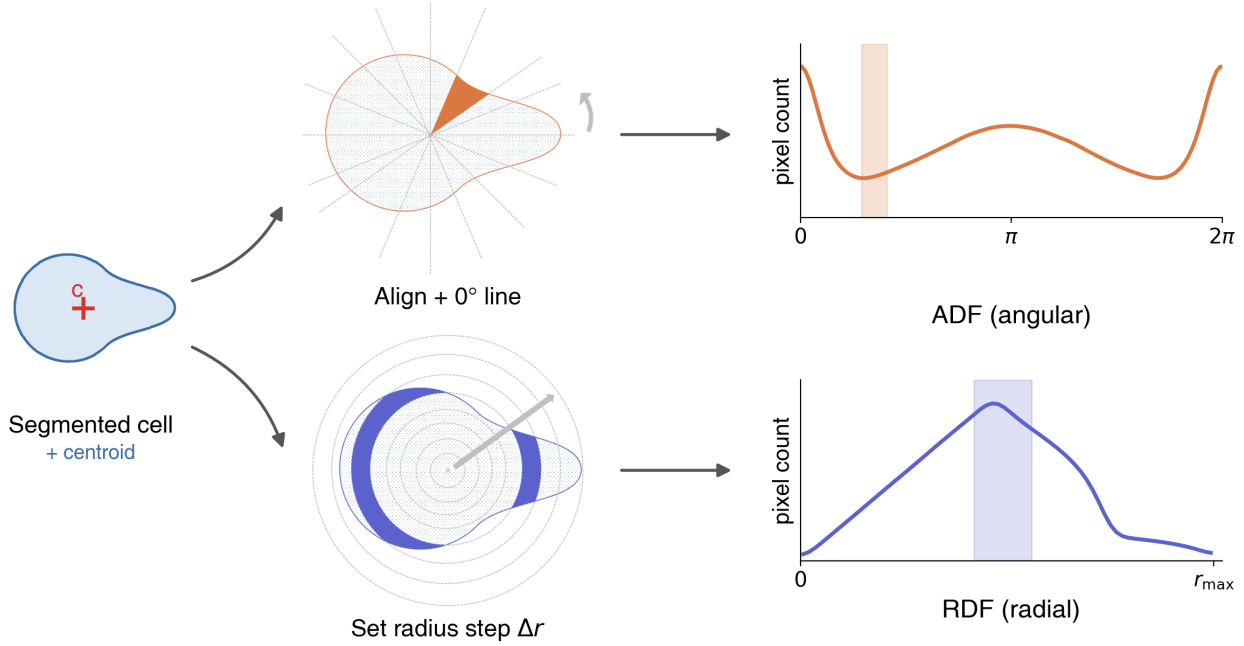


Figure 1: RDF/ADF Extraction: By mapping the segmented cell into polar coordinates (r, θ) centered at its centroid, we quantify the cell's spatial distribution. The Radial Distribution Function (RDF) and Angular Distribution Function (ADF) represent the cell's area as a function of radial distance r and polar angle θ, respectively. Computationally, these are realized as one-dimensional signals by calculating the marginal distributions (histograms) of the foreground pixels along the radial and angular axes (right).

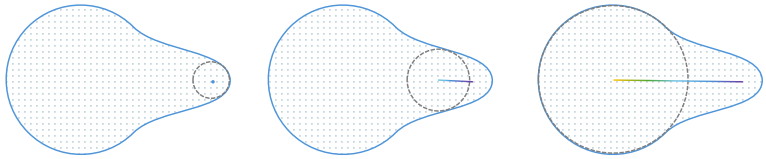


Figure 2: Medial Axis extraction. The medial axis (skeleton) is traced by the centers of the maximal circles that can be inscribed in the cell (dashed). Each such circle touches the boundary, and its radius equals the cell's local thickness; the centers of all these circles form the medial axis (colored curve).

Persistence and embedding.

Each descriptor becomes a one-dimensional signal, which I extract with H_0 *sub-level filtration*: a threshold sweeps from low to high and records where connected components are born (at valleys) and merge (at peaks), giving a persistence *barcode*. Each barcode is turned into a fixed 20×20 *persistence image*, in which strong, long-lived features are emphasized and short-lived features are suppressed. A *convolutional autoencoder (CAE)* then compresses each persistence image into a 32-dimensional code, trained only to reconstruct its input so it uses no cell labels. A CAE is a machine learning tool used to compress the high-dimensional representation of each cell into a lower-dimensional space. Further implementation details are provided in the Appendix.

Finally, these latent embeddings are clustered with k-means (with k chosen by the silhouette score) and visualized in two dimensions with UMAP (Figure 3).

I implemented the whole pipeline in Python using `scikit-image` for image processing, `GUDHI` for persistent homology, and PyTorch for the autoencoder.

2.3 Results

The analysis is still in progress. The following are initial results with the current pipeline settings. Even without ever seeing the stiffness labels, the unsupervised embedding already shows structure that lines up with substrate stiffness. For PMA-stimulated cells ($n = 395$), a UMAP of the ADF latents places soft substrates (1, 10 kPa) and stiff substrates (20, 100 kPa) in largely opposite regions of the embedding (Figure 4).

The main outcome of the term is a working, interpretable, alignment-free morphological descriptor together with an open, reproducible codebase; tuning and further validation are ongoing.

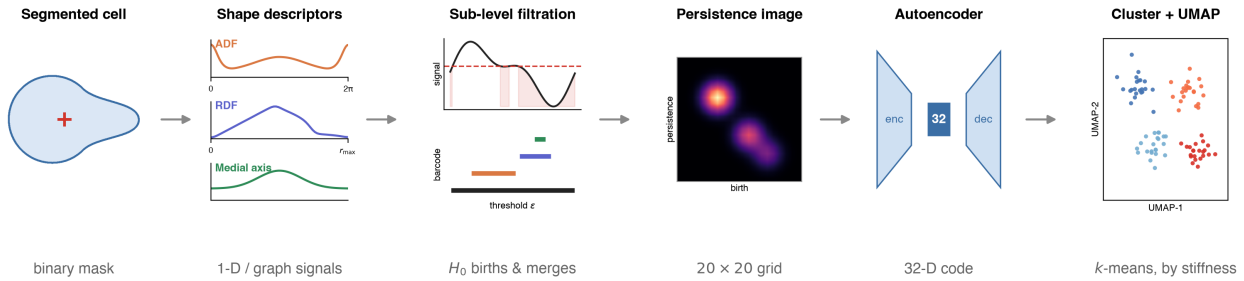


Figure 3: Overview of the pipeline. A segmented cell mask is described by three shape descriptors (ADF, RDF, medial axis), each summarized by H_0 sub-level filtration into a 20×20 persistence image, compressed by a convolutional autoencoder into a 32-dimensional code, and clustered with k-means and visualized with UMAP. For example, using the PMA-stimulated cell dataset ($n = 395$), we pooled the 395 ADF and 395 RDF latent embeddings for k-means clustering, analyzing them both jointly and individually.

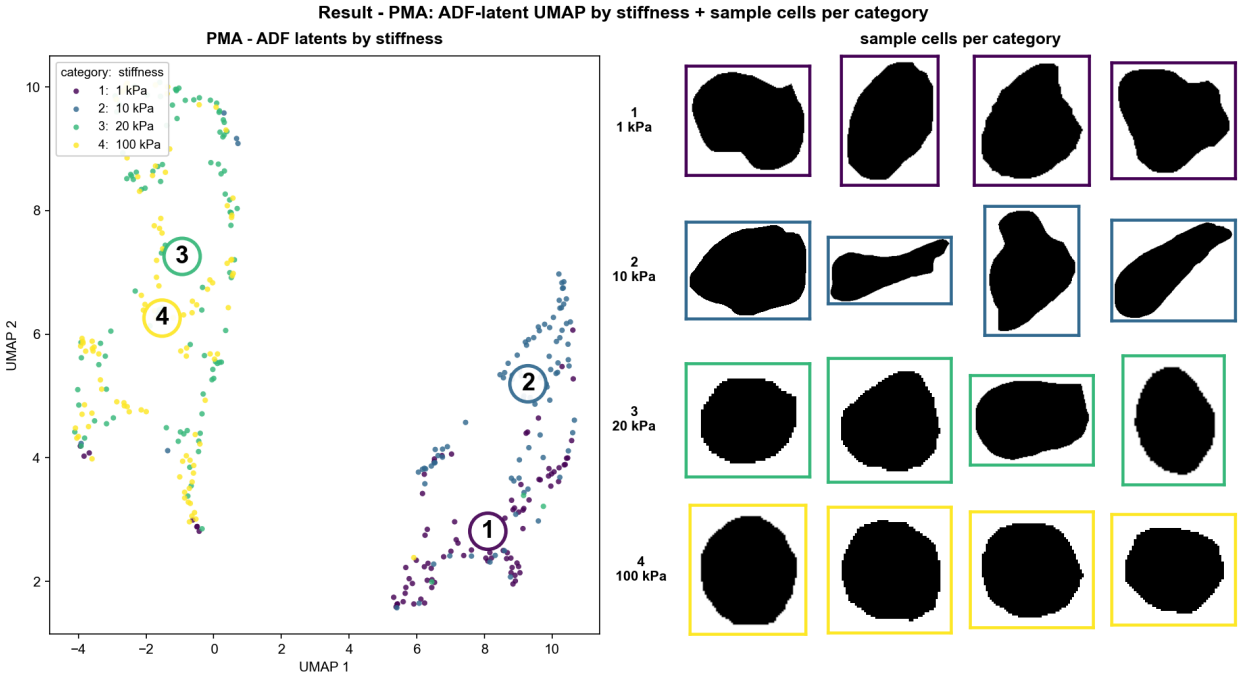


Figure 4: UMAP of the PMA ADF persistence-image latents (32-D autoencoder embedding, $n = 395$), colored by substrate stiffness (1–4 = 1, 10, 20, 100 kPa). Shape alone separates soft (1, 10 kPa) from stiff (20, 100 kPa) substrates.

3 Acknowledgements

I would like to thank Dr. Leah Edelstein-Keshet and Dr. Dhananjay Bhaskar for directing this project mentorship, and support during my WLIURA work term.

4 Appendix

4.1 Convolutional Autoencoder (CAE)

Network Architecture I built a CAE pipeline comprises an encoder for feature extraction and a decoder for spatial reconstruction.

The **encoder** processes single-channel inputs through three convolutional layers (3×3 kernels, padding of 1), progressively increasing the channel depth to 32, 64, and 128. Each convolution is followed by Batch Normalization and a ReLU activation. Spatial dimensions are halved twice via 2×2 max-pooling to form a compact latent representation.

The **decoder** reconstructs the latent space using two transposed convolutions (2×2 kernels, stride of 2), reducing the channel depth to 64 and 32. A final 3×3 convolution maps the features to a single channel, applying a Sigmoid activation to bound the output within $[0, 1]$.

The model is optimized end-to-end to minimize the Mean Squared Error (MSE) loss using the Adam optimizer with a learning rate of 1×10^{-3}.